

SYSTEM DESIGN

URL Shortener

Architecture, Capacity Planning & Infrastructure Sizing

Design Targets

500,000,000 GET requests / day

10,000,000 POST requests / day

1. Functional Requirements

- Create a short URL from a long URL.
- Redirect a short URL to its original URL.
- Support optional custom aliases.
- Support optional URL expiration.

2. Non-Functional Requirements

- High availability.
- Low-latency redirects.
- Horizontal scalability.
- Durable storage.
- Safe concurrent short-code generation.
- Read-heavy workload optimization through caching.

3. API Methods

Method	Purpose	Input	Output
POST /api/v1/urls	Create short URL	url, expiresAt, customAlias	shortUrl, code
GET /{code}	Redirect	code	301/302 + Location
GET /api/v1/urls/{code}	URL metadata	code	originalUrl, timestamps, clicks
DELETE /api/v1/urls/{code}	Delete/disable URL	code	204

4. Back-of-the-Envelope Estimation

Assumptions

- 1 day = 86,400 seconds.
- GET traffic = 500M/day; POST traffic = 10M/day.
- Design for a 10× peak over average traffic.
- Average stored URL record (with indexes/metadata) $\approx$ 500 bytes for an initial estimate.
- A cached redirect response is assumed to average $\approx$ 600 bytes of total application traffic.
- Redis cache entries are estimated at $\approx$ 200 bytes per entry, including key/value and overhead.

Request Rate

Metric	Calculation	Estimate
GET average	500,000,000 / 86,400	$\approx$ 5,787 GET/s
POST average	10,000,000 / 86,400	$\approx$ 116 POST/s
Total average	(500M + 10M) / 86,400	$\approx$ 5,903 requests/s

Metric	Calculation	Estimate
GET peak (10×)	$5,787 \times 10$	$\approx 57,870$ GET/s
POST peak (10×)	116×10	$\approx 1,160$ POST/s

Traffic Volume

- GETs per month (30-day approximation): $500M \times 30 = 15B$.
- POSTs per month: $10M \times 30 = 300M$.
- URLs created per year: $10M \times 365 = 3.65B$.
- Raw logical data: $3.65B \times \sim 500$ bytes ≈ 1.825 TB/year.
- With indexes, WAL, replication, backups, and headroom: a practical planning figure of roughly 3–5 TB/year for primary storage, with additional capacity for replicas/backups.

Redis Estimate

- If 500M URLs were simultaneously cached at ~ 200 bytes each: $500M \times 200$ bytes ≈ 100 GB.
- In practice, not every URL needs to live in Redis — use TTL/LRU-style eviction and cache the hot set.
- A cluster with roughly 150–250 GB usable memory is a reasonable starting point if the hot set is large; measure actual usage before final sizing.

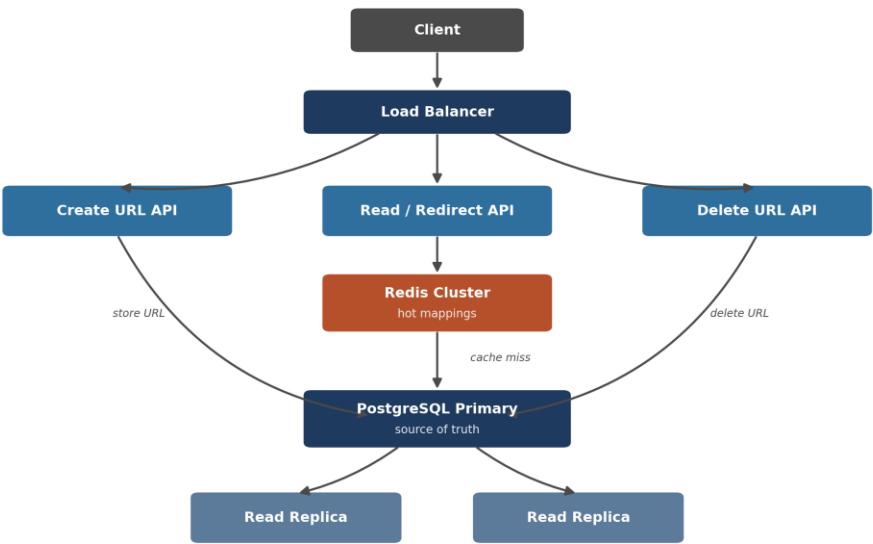
5. Short-Code Generation Methodology

Recommended approach: generate a unique numeric ID and encode it using Base62 (alphabet: 0–9, a–z, A–Z).

Code Length	Number of Combinations
6 characters	$62^6 \approx 56.8$ billion
7 characters	$62^7 \approx 3.52$ trillion
8 characters	$62^8 \approx 218$ trillion

6. High-Level Design

Client requests pass through a load balancer to dedicated Create, Read/Redirect, and Delete services. Reads are served from a Redis cache in front of a PostgreSQL primary with read replicas; writes go directly to the PostgreSQL primary.



7. Server / Infrastructure Sizing

Component	Initial Sizing	Purpose
Load Balancer	2 / managed HA	Distribute traffic and provide high availability
URL API servers	12–16 × 4–8 vCPU, 8–16 GB RAM	Handle peak request load with headroom
Redis	3+ nodes, 4–8 vCPU, 16–32 GB RAM each	Hot URL mappings / low-latency reads
PostgreSQL primary	1 × 8–16 vCPU, 32–64 GB RAM	Durable writes and source of truth
PostgreSQL replicas	2–3	Read scaling and availability

Note: all estimates in this document are intentionally approximate and should be validated with load testing before finalizing capacity plans.